

Name : Krish Ajay Basangar

PRN: 202501040033

3.1.1. Numpy Array Operations

Problem Statement

Write a Python program to create a NumPy array based on user input and display the following:

- The created NumPy array.
- The number of dimensions of the array using ndim
- The shape of the array using shape
- The total number of elements in the array using size

Assume all input elements are valid integers.

Input Format:

- The first line contains two space-separated integers, and , representing the number of rows and the number of columns.
- The next lines contain space-separated integers representing the elements of the array, entered row by row.

Output Format:

- The created NumPy array (printed as a 2D array).
- An integer representing the number of dimensions of the array.
- A tuple representing the shape of the array.
- An integer representing the total number of elements in the array.

Note: Use reshape() function to reshape the input array with the specified number of rows and columns.

Code

```
import numpy as np

# write your code here...
arr=[]
rows,cols=map(int,input().split())
```

```

for _ in range (rows):
row=list(map(int,input().split()))
arr.extend(row)
arr=np.array(arr).reshape(rows,cols)

print(arr)

print(arr.ndim)
print(arr.shape)
print(arr.size)

```

Output

3 4	
1 2 3 4	
5 6 7 8	
9 10 11 12	
[[1 2 3 4]	
5 6 7 8]	0 0
9 10 11 12]]	[]
2	2
(3, 4)	(0, 0)
12	0

3.2.1. Numpy: Matrix Operations

Problem Statement

The given code takes two matrices, , and , as input from the user and converts them into NumPy arrays.

Task:

You are required to compute and display the results of the following matrix operations:

1. **Addition ()**
2. **Subtraction ()**
3. **Element-wise Multiplication ()**

4. **Matrix Multiplication ()**
5. **Transpose of Matrix A**

Input Format:

- The user will input 3 rows for , each containing 3 integers separated by spaces.
- Similarly, the user will input 3 rows for , each containing 3 integers separated by spaces.

Output Format:

The program should display the results of the operations in the following order:

1. The result of Addition.
2. The result of Subtraction.
3. The result of Element-wise Multiplication.
4. The result of Matrix Multiplication.
5. The Transpose of Matrix A.

Code

```
import numpy as np

# Input matrices
print("Enter Matrix A:")
matrix_a = np.array([list(map(int, input().split())) for i in
range(3)])

print("Enter Matrix B:")
matrix_b = np.array([list(map(int, input().split())) for i in
range(3)])

# Addition
print("Addition (A + B):")
print(matrix_a+matrix_b)
# Subtraction
print("Subtraction (A - B):")
print(matrix_a-matrix_b)
# Multiplication (element-wise)
```

```
print("Element-wise Multiplication (A * B):")
print(matrix_a*matrix_b)
# Matrix multiplication (dot product)
print("A dot B:")
print(np.dot(matrix_a,matrix_b))
# Transpose
print("Transpose of A:")
print(matrix_a.T)
```

Output

Enter Matrix A:
1 2 3
4 5 6
7 8 9
Enter Matrix B:
1 1 1
1 1 1
1 1 1
Addition (A+B):
[[2 3 4]
[5 6 7]
[8 9 10]]
Subtraction (A-B):
[[0 1 2]
[3 4 5]
[6 7 8]]
Element-wise Multiplication (A*B):
[[1 2 3]
[4 5 6]
[7 8 9]]
A.dot B:
[[6 6 6]
[15 15 15]
[24 24 24]]
Transpose of A:
[[1 4 7]
[2 5 8]
[3 6 9]]

3.2.2. Numpy: Horizontal and Vertical Stacking of Arrays

Problem Statement

You are given two arrays arr1 and arr2. You need to perform horizontal and vertical stacking operations on them using NumPy.

- **Horizontal Stacking:** Stack the two matrices horizontally (side by side).
- **Vertical Stacking:** Stack the two matrices vertically (one below the other).

Input Format:

- The program should first prompt the user to input two 3x3 arrays.
- Each array consists of 3 rows, and each row contains 3 space-separated integers.
- The user will input the two arrays row by row.

Output Format:

- The program should display the result of the Horizontal Stack (side-by-side stacking) of the two arrays.
- The program should then display the result of the Vertical Stack (one below the other) of the two arrays.

Code

```
import numpy as np

# Input matrices
print("Enter Array1:")
arr1 = np.array([list(map(int, input().split())) for i in range(3)])

print("Enter Array2:")
arr2 = np.array([list(map(int, input().split())) for i in range(3)])

# Perform horizontal stacking (hstack)
print("Horizontal Stack:")
print(np.hstack((arr1, arr2)))

# Perform vertical stacking (vstack)
print("Vertical Stack:")
print(np.vstack((arr1, arr2)))
```

Output

Enter Array1:
1 2 3
4 5 6
7 8 9
Enter Array2:
4 5 6
7 8 9
10 11 12
Horizontal Stack:
[1 2 3 4 5 6]
[4 5 6 7 8 9]
[7 8 9 10 11 12]
Vertical Stack:
[1 2 3]
[4 5 6]
[7 8 9]
[4 5 6]
[7 8 9]
[10 11 12]

3.2.3. Numpy: Custom Sequence Generation

Problem Statement

Write a Python program that takes the following inputs from the user:

- Start value: The starting point of the sequence.
- Stop value: The sequence should end before this value.
- Step value: The increment between each number in the sequence.

The program should then generate a sequence using numpy based on these inputs and print the generated sequence.

Input Format:

- The user will input three integer values: start, stop, and step, each on a new line.

Output Format:

- The program should print the generated sequence based on the input values.

Code

```
import numpy as np

# Take user input for the start, stop, and step of the sequence
start = int(input())
stop = int(input())
step = int(input())

# Generate the sequence using np.arange()

# Print the generated sequence
print(np.arange(start, stop, step))
```

Output

3	10
10	20
2	30
[3. 5. 7. 9.]	[10.]

3.2.4. Numpy: Arithmetic and Statistical Operations.

Mathematical Operations ,Bitwise Operations

Problem Statement

You are given two arrays A and B. Your task is to complete the function `array_operations`, which will convert these lists into NumPy arrays and perform the following operations:

1. Arithmetic Operations:

- Compute the element-wise sum, difference, and product of the two arrays.

2. Statistical Operations:

- Calculate the mean, median, and standard deviation of array A.

3. Bitwise Operations:

- Perform bitwise AND, bitwise OR, and bitwise XOR on the arrays (ex: $A_i \text{ OR } B_i$).

Input Format:

- The first line contains space-separated integers representing the elements of array A.
- The second line contains space-separated integers representing the elements of array B.

Output Format:

- For each operation (arithmetic, statistical, and bitwise), print the results in the specified format as shown in sample test cases.

Code

```
import numpy as np

def array_operations(A, B):

    # Convert A and B to NumPy arrays
    A = np.array(A)
    B = np.array(B)

    # Arithmetic Operations
    sum_result = A+B
    diff_result = A-B
    prod_result = A*B

    # Statistical Operations
    mean_A = np.mean(A)
    median_A = np.median(A)
    std_dev_A = np.std(A)

    # Bitwise Operations
    and_result = np.bitwise_and(A,B)
    or_result = np.bitwise_or(A,B)
    xor_result = np.bitwise_xor(A,B)

    # Output results with one space between each element
    print("Element-wise Sum:", ' '.join(map(str, sum_result)))
    print("Element-wise Difference:", ' '.join(map(str, diff_result)))
    print("Element-wise Product:", ' '.join(map(str, prod_result)))
    print(f"Mean of A: {mean_A}")
    print(f"Median of A: {median_A}")
    print(f"Standard Deviation of A: {std_dev_A}")
    print("Bitwise AND:", ' '.join(map(str, and_result)))
```

```
print("Bitwise OR:", ' '.join(map(str, or_result)))
print("Bitwise XOR:", ' '.join(map(str, xor_result)))

A = list(map(int, input().split())) # Elements of array A
B = list(map(int, input().split())) # Elements of array B
array_operations(A, B)
```

Output

1 2 3 4
5 6 7 8
Element-wise Sum: 6 8 10 12
Element-wise Difference: -4 -4 -4 -4
Element-wise Product: 5 12 21 32
Mean of A: 2.5
Median of A: 2.5
Standard Deviation of A: 1.118033988749895
Bitwise AND: 1 2 3 0
Bitwise OR: 5 6 7 12
Bitwise XOR: 4 4 4 12

3.2.5. Numpy: Copying and Viewing Array

Problem Statement

The given code takes a list of integers as input and converts it into a NumPy array. Your task is to complete the code by:

- Creating a view of the original_array and assigning it to view_array.
- Creating a copy of the original_array and assigning it to copy_array.

After completing these steps, observe how modifying the view affects the original_array, while modifying the copy does not.

Input Format:

- A single line of space-separated integers.

Output Format:

- After modifying the view:

Original array after modifying view: <original_array>
View array: <view_array>

- After modifying the copy:

Original array after modifying copy: <original_array>
Copy array: <copy_array>

Code

```
import numpy as np

inputlist = list(map(int, input().split(" ")))

# Original array
original_array = np.array(inputlist)

# Create a view
view_array = original_array.view()

# Create a copy
copy_array = original_array.copy()

# Modify the view
view_array[0] = 99
print("Original array after modifying view:", original_array)
print("View array:", view_array)

# Modify the copy
copy_array[1] = 88
print("Original array after modifying copy:", original_array)
print("Copy array:", copy_array)
```

Output

```
10 20 30 40 50 60 70 80
```

```
Original array after modifying view: [99 20 30 40 50 60 70 80]
```

```
View array: [99 20 30 40 50 60 70 80]
```

```
Original array after modifying copy: [99 20 30 40 50 60 70 80]
```

```
Copy array: [10 88 30 40 50 60 70 80]
```

3.2.6. Numpy: Searching , Sorting, Counting, Broadcasting

Problem Statement

The given code in the editor takes a single array, array1, as space-separated integers as input from the user.

Additionally, it takes the following inputs:

- search_value: The value to search for in the array.
- count_value: The value to count its occurrences in the array.
- broadcast_value: The value to add for broadcasting across the array.

You need to complete the code to perform the following operations:

- 1. Searching:** Find the indices where search_value appears in array1 and print these indices.
- 2. Counting:** Count how many times count_value appears in array1 and print the count.
- 3. Broadcasting:** Add broadcast_value to each element of array1 using broadcasting, and print the resulting array.
- 4. Sorting:** Sort array1 in ascending order and print the sorted array.

Input Format:

1. A single line containing space-separated integers representing array1.
2. An integer search_value represents the value to search for in the array.
3. An integer count_value represents the value to count in the array.
4. An integer broadcast_value represents the value to add to each element of the array.

Output Format:

1. The indices where search_value occurs in array1.
2. The count of occurrences of count_value in array1.
3. The array after adding the broadcast_value to each element.
4. The sorted array.

Code

```
import numpy as np
```

```

# Input array from the user
array1 = np.array(list(map(int, input().split()))))

# Searching
search_value = int(input("Value to search: "))
count_value = int(input("Value to count: "))
broadcast_value = int(input("Value to add: "))

# Find indices where value matches in array1
indices=np.where(array1==search_value)[0]
print(indices)
# Count occurrences in array1
count=np.count_nonzero(array1==count_value)
print(count)
ba=array1+broadcast_value
print(ba)
# Broadcasting addition
sorted_array=np.sort(array1)
print(sorted_array)
# Sort the first array

```

Output

1 1 1 2 2 2	1 2 3 4 5
Value to search: 1	Value to search: 6
Value to count: 2	Value to count: 6
Value to add: 2	Value to add: 6
[0 1 2]	[]
3	0
[3 3 3 4 4 4]	[7 8 9 10 11]
[1 1 1 2 2 2]	[1 2 3 4 5]

3.2.7. Student Data Analysis and Operations

Problem Statement

Write a Python program that takes the file name of a CSV file containing student details, including roll numbers and their marks in three subjects as input, reads the data, and performs the following operations:

- **Print all student details:** Display the complete details of all students, including roll numbers and marks for all subjects.
- **Find total students:** Determine the total number of students in the dataset.

- **Print all student roll numbers:** Extract and print the roll numbers of all students.
- **Print Subject 1 marks:** Extract and print the marks of all students in Subject 1.
- **Find minimum marks in Subject 2:** Identify the lowest marks in Subject 2.
- **Find maximum marks in Subject 3:** Identify the highest marks in Subject 3.
- **Print all subject marks:** Display the marks of all students for each subject.
- **Find total marks of students:** Compute the total marks for each student across all subjects.
- **Find the average marks of each student:** Compute the average marks for each student.
- **Find average marks of each subject:** Compute the average marks for all students in each subject.
- **Find average marks of Subject 1 and Subject 2:** Compute the average marks for Subject 1 and Subject 2.
- **Find average marks of Subject 1 and Subject 3:** Compute the average marks for Subject 1 and Subject 3.
- **Find the roll number of the student with maximum marks in Subject 3:** Identify the student with the highest marks in Subject 3 and print their roll number.
- **Find the roll number of the student with minimum marks in Subject 2:** Identify the student with the lowest marks in Subject 2 and print their roll number.
- **Find the roll number of students who scored 24 marks in Subject 2:** Identify students who obtained exactly 24 marks in Subject 2 and print their roll numbers.
- **Find the count of students who got less than 40 marks in Subject 1:** Count the number of students who scored less than 40 marks in Subject 1.
- **Find the count of students who got more than 90 marks in Subject 2:** Count the number of students who scored more than 90 marks in Subject 2.
- **Find the count of students who scored ≥ 90 in each subject:** Count the number of students who scored 90 or more marks in each subject.
- **Find the count of subjects in which each student scored ≥ 90 :** Determine how many subjects each student scored 90 or more marks in.
- **Print Subject 1 marks in ascending order:** Sort and print the marks of students in Subject 1 in ascending order.
- **Print students who scored between 50 and 90 in Subject 1:** Display students who scored marks between 50 and 90 in Subject 1.
- **Find index positions of students who scored 79 in Subject 1:** Identify the index positions of students who scored exactly 79 marks in Subject 1.

Note: Fill in the missing code to perform the above-mentioned operations.

Code

```
import numpy as np

a = np.loadtxt("Sample.csv", delimiter=',', skiprows=1)

# 1. Print all student details
print("All student Details:\n", a )

# 2. print total students

print("Total Students:", len(a))
```

```

# 3. Print all student Roll numbers
print("All Student Roll Nos", a[:,0] )

# 4. Print subject 1 marks
print("Subject 1 Marks", a[:, 1])

# 5. print minimum marks of Subject 2
print("Min marks in Subject 2", np.min(a[:, 2]) )

# 6. print maximum marks of Subject 3
print("Max marks in Subject 3", np.max(a[:, 3:]) )

# 7. Print All subject marks
print("All subject marks:", (a[:, 1:]))

# 8. print Total marks of students
print("Total Marks", np.sum(a[:, 1:],axis=1) )

# 9. print average marks of each student
print(np.round(np.mean(a[:, 1:], axis=1),1))

# 10. print average marks of each subject
print("Average Marks of each subject", np.mean(a[:, 1:], axis=0) )

# 11. print average marks of S1 and S2
print("Average Marks of S1 and S2", np.mean(a[:, 1:3], axis=0) )

# 12. print average marks of S1 and S3
print("Average Marks of S1 and S3", np.mean(a[:, [1, 3]], axis=0) )

# 13. print Roll number who got maximum marks in Subject 3
max_sub3_index = np.argmax(a[:, 3])
print("Roll no who got maximum marks in Subject 3", a[max_sub3_index,
0] )

# 14. print Roll number who got minimum marks in Subject 2
min_sub2_index = np.argmin(a[:,2])
print("Roll no who got minimum marks in Subject 2", a[min_sub2_index,
0] )

# 15. print Roll number who got 24 marks in Subject 2

print(f"Roll no who got 24 marks in Subject 2 [{a[a[:, 2] ==
24][:,0]}]" )

# 16. print count of students who got marks in Subject 1 < 40
count_sub1_lt_40 = np.sum(a[:, 1] < 40)

```

```
print("Count of students who got marks in Subject 1 < 40",  
count_sub1_lt_40 )
```

Output

```
All student Details:  
·[[301..67..77..88.]  
·[302..78..88..77.]  
·[303..45..56..89.]  
·[304..88..98..45.]  
·[305..78..88..99.]  
·[306..68..78..36.]  
·[307..79..12..45.]  
·[308..46..45..77.]  
·[309..89..32..88.]  
·[310..79..24..33.]]  
Total Students: 10  
All Student Roll Nos [301..302..303..304..305..306..307..308..309..310.]  
Subject 1 Marks [67..78..45..88..78..68..79..46..89..79.]  
Min marks in Subject 2 12.0  
Max marks in Subject 3 99.0  
All subject marks: [[67..77..88.]  
·[78..88..77.]  
·[45..56..89.]  
·[88..98..45.]  
·[78..88..99.]  
·[68..78..36.]  
·[79..12..45.]  
·[46..45..77.]  
·[89..32..88.]  
·[79..24..33.]]  
Total Marks [232..243..190..231..265..182..136..168..209..136.]
```



```

[77.3·81.··63.3·77.··88.3·60.7·45.3·56.··69.7·45.3]
Average Marks of each subject [71.7·59.8·67.7]
Average Marks of S1 and S2 [71.7·59.8]
Average Marks of S1 and S3 [71.7·67.7]
Roll no who got maximum marks in Subject 3 305.0
Roll no who got minimum marks in Subject 2 307.0
Roll no who got 24 marks in Subject 2 [[310.]]
Count of students who got marks in Subject 1 < 40 0
Count of students who got marks in Subject 2 > 90 1
Count of students in each subject who got marks >= 90: [0·1·1]
Roll no: [301.·302.·303.·304.·305.·306.·307.·308.·309.·310.]
Count of subjects in which student got marks >= 90: [0·0·0·1·1·0·0·0·0·0]
[45.·46.·67.·68.·78.·78.·79.·79.·88.·89.]
[[301.··67.··77.··88.]
·[302.··78.··88.··77.]
·[304.··88.··98.··45.]
·[305.··78.··88.··99.]
·[306.··68.··78.··36.]
·[307.··79.··12.··45.]
·[309.··89.··32.··88.]
·[310.··79.··24.··33.]]
[[301.··67.··77.··88.]
·[302.··78.··88.··77.]
·[303.··45.··56.··89.]
·[304.··88.··98.··45.]
·[305.··78.··88.··99.]
·[306.··68.··78.··36.]
·[307.··79.··12.··45.]
·[308.··46.··45.··77.]
·[305.··78.··88.··99.]
·[306.··68.··78.··36.]
·[307.··79.··12.··45.]
·[308.··46.··45.··77.]
·[309.··89.··32.··88.]
·[310.··79.··24.··33.]]
(array([6, 9], dtype=int32),)

```

